

TCP Congestion Window in NS-3

UNDERSTANDING TCP CWND
BEHAVIOR USING NS-3
SIMULATIONS



TCP Congestion Window (cwnd)

- TCP state variable that limits how many bytes (or packets) a TCP sender can send before waiting for acknowledgments (ACKs).
- It grows/shrinks based on network feedback (ACKs, packet loss).
- Prevents overloading the network.
- Every TCP variant (Tahoe, Reno, NewReno, Cubic, BBR, etc.) manipulates cwnd differently to balance speed and fairness.



TCP in NS-3

- NS-3 provides multiple TCP variants (Reno, NewReno, Cubic, Westwood, etc.).
- **CongestionWindow attribute:** Each TCP socket has this attribute, which can be **traced** to record its changes during simulation.
- Applications:
 - BulkSendApplication → sends a continuous stream of data using TCP.
 - PacketSinkApplication → receives the data and acknowledges it
- **Topologies:** Often use simple **Point-to-Point links** to isolate TCP behavior.

Example TCP Topology in NS-3

- Node 0 ----(10 Mbps, 4 ms)---- Node 1 ----(10 Mbps, 4 ms)---- Node 2
 - **Node 0:** TCP sender (with BulkSendApplication).
 - **Node 1:** Router (just forwards packets).
 - **Node 2:** TCP receiver (with PacketSink).
 - **Purpose:** To observe TCP cwnd behavior when traffic flows through intermediate nodes with bandwidth and delay limits.



TCP Applications

- **BulkSendApplication (Node 0)**: Generates a continuous stream of TCP traffic → this stresses TCP and triggers congestion control.
- **PacketSinkApplication (Node 2)**: Receives the packets and acknowledges them back.
- Together, they create a **TCP connection** across the network.

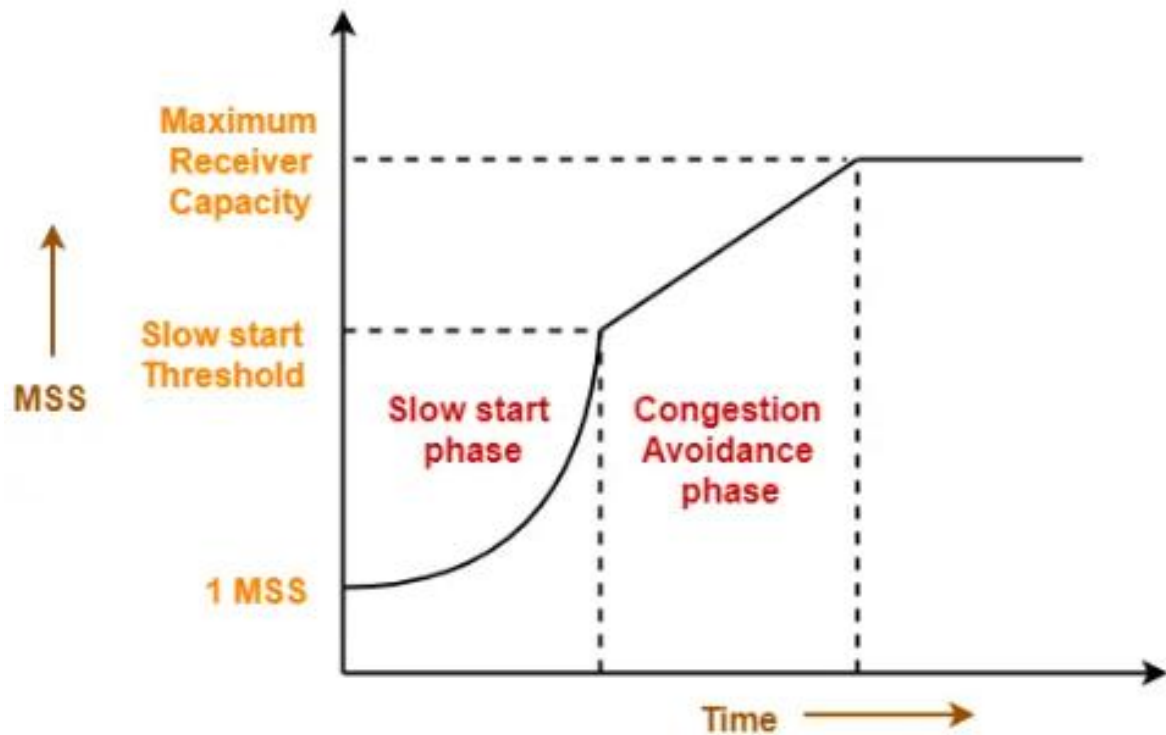
Congestion Window Tracing

- In ns-3, cwnd can be **traced** by connecting to the **CongestionWindow** attribute of the TCP socket.
- This callback **CwndTracer** writes **time vs cwnd size** into a log file.

```
Config::ConnectWithoutContext(  
    "/NodeList/0/$ns3::TcpL4Protocol/SocketList/0/CongestionWindow",  
    MakeCallback(&CwndTracer));
```

- This is the **core mechanism** that allows us to monitor TCP behavior.

TCP congestion window (cwnd) evolution



- TCP cwnd typically evolves in phases:
 - **Slow Start:** cwnd grows exponentially (doubles each RTT).
 - **Congestion Avoidance:** linear growth (additive increase).
 - **On Loss:** cwnd drops suddenly (multiplicative decrease).
- The output log is a **time vs cwnd** table, which can later be plotted.



Exponential Growth (Slow Start)

- Exponential Growth (Slow Start)
 - At the beginning of a TCP connection, cwnd starts small (e.g., 1 MSS – Maximum Segment Size).
 - For **every ACK received**, cwnd increases by **1 MSS**.
 - This means in one RTT, cwnd roughly **doubles** → hence *exponential growth*.



Linear Growth (Congestion Avoidance)

- Linear Growth (Congestion Avoidance)
 - After cwnd reaches a threshold (**ssthresh**), TCP enters *Congestion Avoidance*.
 - Now, cwnd increases by about **1 MSS per RTT** (additive increase).
 - Growth is **slower and linear** → prevents network overload.

Sharp Drops (Multiplicative Decrease)

- Sharp Drops (Multiplicative Decrease)
 - When **packet loss** occurs (detected via timeout or 3 duplicate ACKs), TCP assumes congestion.
 - cwnd is **cut down** (usually halved or reset to 1 in older TCP like Tahoe).
 - This sudden drop avoids flooding the congested network.



Visualization & Analysis

- **NetAnim**: Visualize packet transfers across nodes.
- **cwnd logs**: Can be plotted using **Gnuplot** or **Matplotlib**.
- The plot will show the **classic TCP sawtooth pattern** → growth → drop → growth.